My Courses

WY 2-1-1 Unrolled List

参考用时估计: 4小时

【题目描述】

“展开的链表”（unrolled linked list）是由链表衍生出的一种线性结构，它的每个节点(node)本身就是一个向量/数组，相邻节点之间按照链表的方式依次连接。每个节点分别对应一段连续的内存，数据在其中顺序存放，因此在每个节点内部可以O(1)时间内循秩访问。

当然，如果节点的数目过多，查找操作的时间复杂度，将和普通链表循位置访问几乎相同(最坏为线性); 如果目标元素所属的节点过大，插入操作的时间复杂度，和普通向量的插入操作几乎相同（最坏为线性）。幸运的是，”展开的链表“可以通过一些适当的策略兼顾两类操作，时间复杂度总能控制在O(sqrt(n))。

具体来说，我们需要在节点的数量与规模之间取得一种平衡，比如说使二者总在O(sqrt(n))以内。

一种简单的实现方式是，给每一段包含的元素数目设定一个固定的上限，当插入操作使得这一段内包含的元素数目达到这个上限时，就对其进行分裂，变为两个链表节点。

现在给出一个分块链表的程序框架，请你补全它，并进行黑盒测试。注意，如果不使用程序框架、或者使用其他数据结构，即使完成相应功能并通过黑盒测试，也不能获取分数。

程序框架下载: unrolled.cpp (attachment/7b97/7b9743853a774a80495608dee6beaa802a185a99.cpp)

黑盒测试的要求为:

假设你有一个存储正整数的链表，初始为空。随后按照下面的格式处理一系列操作。

【输入格式】

首先输入一个正整数，表示接下来进行的操作次数（接下来要输入的行数）。

接下来每行输入三个整数，代表一次操作。

每一行的第一个整数表示操作类型，1为插入，2为删除，3为查询。第二、三个整数是操作的参数。具体含义如下：

1. 对于插入操作，一行三个整数，第二个整数表示插入的位置，即插入之后，新插入的整数所在的位置。如果要插入的位置不存在，那么将元素插入到最后。
2. 对于删除操作，一行三个整数，第二个整数表示要删除的元素所在的位置。如果要删除的位置不存在，那么什么都不做。第三个整数总为0，需要忽略它。
3. 对于查询操作，一行三个整数，第二个整数表示要查询的元素所在的位置。如果要查询的位置不存在，那么返回-1。第三个整数总为0，需要忽略它。

关于整数和位置的对应关系，具体来说，如果链表内当前有m个元素，那么：

- [0,m]这m+1个整数依次对应当前存在的m+1个插入位置
- [0,m-1]这m个整数依次对应当前存在的m个查询位置/删除位置。

【输出格式】

对于每个查询操作输出一行，表示查询的位置所对应的元素。如果该位置不存在，输出-1。

【样例输入】

```
10
1 0 1
1 1 2
1 2 3
1 6 4
1 4 5
2 -1 0
2 9 0
2 2 0
3 2 0
3 9 0
```

【样例输出】

```
4
-1
```

【样例解释】

每次插入/删除操作后，整个数据结构中的内容依次变为:

[1]->[1,2]-> [1,2,3]->[1,2,3,4]->[1,2,3,4,5]->[1,2,3,4,5]->[1,2,3,4,5]->[1,2,4,5]

【时空限制】

时间限制: 1.5s 空间限制: 128MB

【数据范围】

保证0<n<=500000。

保证操作类型的取值为1，2或3，插入操作的位置在int范围内，插入操作的数值为int范围内的正整数。删除操作的位置在int范围内。查询操作的位置在int范围内。

保证删除操作和查询操作的“第三个整数”为0.

【提示】

在本地构建测例进行测试，为实验框架中的M选择合适的数值。

UI powered by Twitter Bootstrap (http://getbootstrap.com/).
Tsinghua Online Judge is designed and coded by Li Ruizhe.
For all suggestions and bug reports, contact oj[at]liruizhe[dot]org.